TOGAF ADM - Die 8 Phasen im Detail erklärt

🎯 Phase A: Architecture Vision

"Was wollen wir eigentlich erreichen?"

Analogie: Du träumst von einem Traumhaus und skizzierst erste Ideen auf einer Serviette.

Was passiert hier:

- Du verstehst das **Geschäftsproblem**: "Unsere Website ist zu langsam, wir verlieren Kunden"
- Du definierst die **Vision**: "Wir wollen eine schnelle, skalierbare Cloud-Plattform"
- Du holst die **wichtigen Leute** an Bord (CEO, CTO, Abteilungsleiter)
- Du erstellst ein grobes **Zielbild**

Outputs:

- Architecture Vision Document (1-2 Seiten)
- High-Level Requirements ("System muss 10.000 Nutzer gleichzeitig bedienen")
- Stakeholder-Map (Wer ist beteiligt? Wer entscheidet?)

Beispiel:

> "Unsere Bank will eine moderne Banking-App. Vision: Kunden können in 30 Sekunden ein Konto eröffnen, nicht wie jetzt in 3 Tagen."

🏢 Phase B: Business Architecture

"Wie funktioniert unser Geschäft?"

Analogie: Du planst die Raumaufteilung - Küche, Wohnzimmer, Büro, etc.

Was passiert hier:

- Du modellierst **Geschäftsprozesse**: Wie läuft eine Bestellung ab? Vom Warenkorb bis zur Lieferung.
- Du definierst **Business Services**: "Kundenregistrierung", "Bestellabwicklung", "Retouren"
- Du verstehst die **Organisation**: Welche Abteilungen gibt es? Wer macht was?

Wichtige Artefakte:

- **Business Process Models** (Flowcharts der Prozesse)
- **Organizational Charts** (Organigramme)
- **Business Capability Map** (Was kann das Unternehmen?)

Beispiel:

```
Prozess: Online-Bestellung
1. Kunde legt Produkt in Warenkorb
```

```
2. Kunde geht zur Kasse
3. Zahlungsdaten werden validiert
4. Bestellung wird ans Lager geschickt
5. Versandbenachrichtigung geht raus
```

AS-IS vs. TO-BE:

- **AS-IS** (Ist-Zustand): "Bestellung dauert 2 Minuten, viele brechen ab"
- **TO-BE** (Soll-Zustand): "One-Click-Bestellung in 10 Sekunden"

💾 Phase C: Information Systems Architectures

"Welche Daten und Anwendungen brauchen wir?"

Diese Phase hat **2 Teile**:

C1: Data Architecture - "Welche Daten fließen wo?"

Analogie: Du planst die Wasser- und Stromleitungen im Haus.

Was passiert:

- Du identifizierst **Datenquellen**: Kundendatenbank, Produktkatalog, Bestellhistorie
- Du definierst **Datenflüsse**: "Kundendaten fließen von der Registrierung ins CRM"
- Du planst **Datenmanagement**: Wo werden Daten gespeichert? Wie lange? Wer darf darauf zugreifen?

Beispiel:

```
Datenfluss einer Bestellung:
Webshop → Order-Datenbank → Warehouse-System → Versanddienstleister
         ↓
    Customer-DB (für Profil-Updates)
```

C2: Application Architecture - "Welche Software-Systeme brauchen wir?"

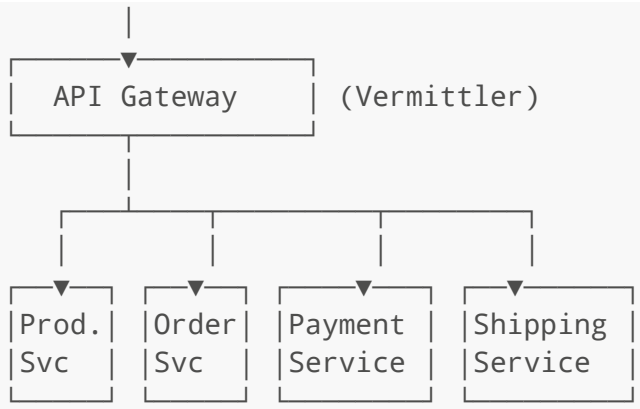
Analogie: Du wählst die Geräte aus - Herd, Kühlschrank, Waschmaschine.

Was passiert:

- Du kartierst **bestehende Anwendungen**: SAP, Salesforce, eigene Legacy-Software
- Du definierst **neue Anwendungen**: Mobile App, API-Gateway, Microservices
- Du zeigst **Beziehungen**: "Mobile App kommuniziert mit Backend-API"

Beispiel - E-Commerce:

```
┌──────────────┐
│  Web-Shop    │ (Frontend)
└──────┬───────┘
```



Technology Mapping: Du ordnest zu: "Payment Service → Business Function: Zahlungsabwicklung"

⚙ Phase D: Technology Architecture

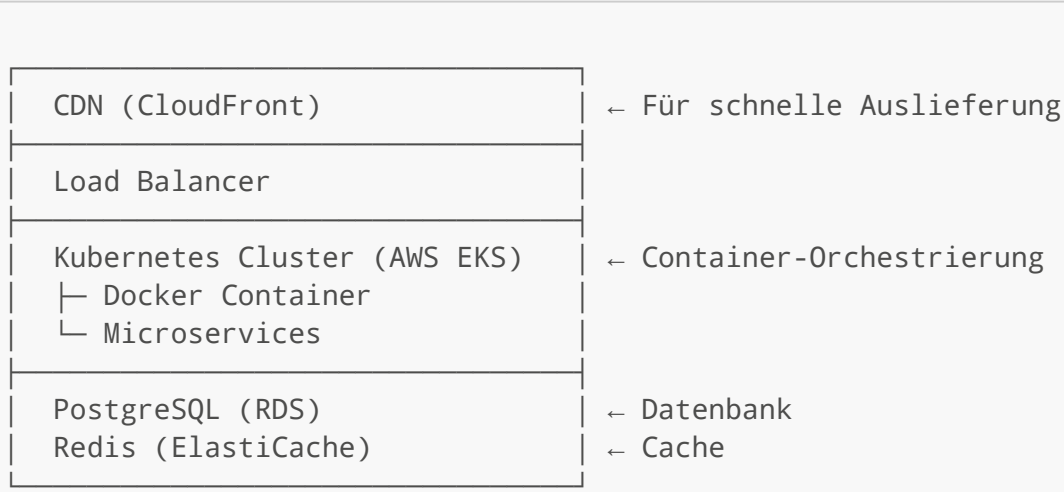
"Welche Technik setzen wir ein?"

Analogie: Du wählst Baumaterialien - Beton, Holz, Ziegel, Dämmung.

Was passiert:

- Du definierst die **technische Infrastruktur**: Server, Cloud, Netzwerk
- Du wählst **Technologie-Stack**:
  - Frontend: React
  - Backend: Java Spring Boot
  - Datenbank: PostgreSQL
  - Cloud: AWS
- Du planst **Security & Compliance**: Firewall, Verschlüsselung, GDPR

Beispiel - Technology Stack:



Deliverables:

- Technology Stack Document
- Infrastructure Diagrams

- Security Architecture

🔍 Phase E: Opportunities & Solutions

"Was sind unsere Optionen zur Umsetzung?"

Analogie: Du vergleichst verschiedene Baufirmen und Angebote.

Was passiert:

- **Gap Analysis**: "Was haben wir? Was fehlt uns?"
- **Portfolio-Analyse**: "Welche Projekte müssen wir starten?"
- **Roadmap erstellen**: "In welcher Reihenfolge bauen wir?"

Beispiel - Gap Analysis:

Komponente	AS-IS	TO-BE	Gap
Frontend	Monolith	React SPA	Neu entwickeln
Backend	Legacy Java	Microservices	Refactoring
Datenbank	Oracle	PostgreSQL	Migration
Cloud	On-Premises	AWS	Cloud Migration

Work Packages (Arbeitspakete):

1. **WP1**: API Gateway aufsetzen (2 Monate, Team A)
2. **WP2**: Microservices entwickeln (4 Monate, Team B)
3. **WP3**: Datenbank migrieren (3 Monate, Team C)

Implementierungs-Roadmap:

```
Q1 2024: Foundation (Cloud-Setup, API Gateway)
Q2 2024: Core Services (Auth, Customer, Order)
Q3 2024: Advanced Features (Payment, Shipping)
Q4 2024: Testing & Go-Live
```

🚚 Phase F: Migration Planning

"Wie kommen wir vom alten zum neuen System?"

Analogie: Du planst den Umzug - was packst du zuerst, was bleibt noch in der alten Wohnung?

Was passiert:

- Du wählst eine **Migrationsstrategie**
- Du planst **Risiken und Ressourcen**
- Du managst **Abhängigkeiten**

3 Haupt-Migrationsansätze:

1. **Big Bang** 🎆

- Alles auf einmal umstellen
- **Pro**: Schnell fertig
- **Contra**: Hohes Risiko, keine Rückzugsmöglichkeit
- **Beispiel**: "Samstagnacht schalten wir das alte System ab, Sonntag läuft das neue"

2. **Phased (Schrittweise)** 📊

- In Etappen migrieren
- **Pro**: Kontrollierbares Risiko
- **Contra**: Dauert länger
- **Beispiel**: "Monat 1: Deutschland, Monat 2: Österreich, Monat 3: Schweiz"

3. **Parallel** ⚖️

- Beide Systeme laufen gleichzeitig
- **Pro**: Sicherheitsnetz, jederzeit zurück möglich
- **Contra**: Doppelter Aufwand
- **Beispiel**: "6 Monate laufen alt und neu parallel, dann schalten wir alt ab"

Strangler Pattern (Beliebt bei Legacy-Systemen):

```
Alt: [████████████████████] 100%
     ↓ Monat 1
     [██████████████▒▒▒▒▒▒] 80% Alt, 20% Neu
     ↓ Monat 6
     [████████▒▒▒▒▒▒▒▒▒▒▒▒] 40% Alt, 60% Neu
     ↓ Monat 12
     [▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒] 0% Alt, 100% Neu ✓
```

Risikomanagement:

- Was kann schiefgehen? (Datenverlust, Ausfall, Performance-Probleme)
- Backup-Pläne
- Rollback-Strategie

👁️ Phase G: Implementation Governance

"Läuft die Umsetzung nach Plan?"

Analogie: Der Bauinspektor kommt regelmäßig vorbei und prüft.

Was passiert:

- **Monitoring**: Du überwachst den Fortschritt
- **Kontrolle**: Werden Standards eingehalten?

- **Änderungsmanagement**: Jemand will etwas ändern → Muss geprüft werden
- **Qualitätssicherung**: Code Reviews, Testing, Security Audits

Architecture Review Board (ARB): Ein Gremium trifft sich wöchentlich:

- Prüft: "Wird nach den Architektur-Vorgaben entwickelt?"
- Entscheidet: "Dürfen wir von der Architektur abweichen?"
- Eskaliert: "Dieses Problem muss der CTO entscheiden"

Beispiel - Governance Gate:

```
Team A: "Wir wollen MongoDB statt PostgreSQL verwenden"
       ↓
ARB-Meeting: Diskussion
       ↓
Entscheidung: "Nein, wir bleiben bei PostgreSQL aus
               Konsistenzgründen"
```

KPIs (Erfolgsmessung):

- Migration Progress: 60% der Services migriert
- Performance: API Response Time < 200ms ✓
- Kosten: Budgetrahmen eingehalten ✓
- Risiken: 3 kritische Issues offen ⚠

🔄 Phase H: Architecture Change Management

"Wie halten wir die Architektur aktuell?"

Analogie: Regelmäßige Wartung des Hauses - Heizung checken, Dach reparieren.

Was passiert:

- **Continuous Monitoring**: System läuft, aber wie gut?
- **Change Requests**: "Wir brauchen ein neues Feature!"
- **Architektur-Updates**: Technologien ändern sich, Architektur muss angepasst werden

Aktivitäten:

1. **Performance Monitoring**

- Ist das System noch performant?
- Skaliert es wie erwartet?
- Gibt es Bottlenecks?

2. **Change Request Management**

```
Request: "Wir wollen AI-Chatbot integrieren"
        ↓
Bewertung: Impact auf bestehende Architektur?
        ↓
Klein → Direktes Go
Groß  → Zurück zu Phase A (neuer ADM-Zyklus)
```

3. **Architektur-Aktualisierung**

- Dokumentation auf dem neuesten Stand halten
- Neue Best Practices einarbeiten
- Technologie-Stack modernisieren

Beispiel:

```
Jahr 1: System läuft stabil
Jahr 2: Neue EU-Regulierung → Architektur anpassen
Jahr 3: Neue Cloud-Services verfügbar → Evaluation
Jahr 4: Legacy-Komponente wird End-of-Life → Migration planen
```

🔁 Der iterative Charakter

WICHTIG: TOGAF ist **kein Wasserfall**!

Du springst zwischen den Phasen hin und her:

```
A → B → C → D → E → "Moment, in Phase B haben wir was vergessen"
                → zurück zu B → weiter mit E → F → G → H
                                                    ↓
                    "Neues Projekt" → zurück zu A
```

🏷 Zusammenfassung - Die 8 Phasen auf einen Blick

Phase	Name	Kernfrage	Output
A	Vision	Was wollen wir?	Vision Document
B	Business	Wie arbeiten wir?	Process Models
C	Information Systems	Welche Systeme & Daten?	Data & App Architecture
D	Technology	Welche Technik?	Tech Stack
E	Opportunities	Welche Optionen?	Roadmap, Gaps
F	Migration	Wie kommen wir dorthin?	Migration Plan

Phase	Name	Kernfrage	Output
G	Governance	Läuft alles?	Monitoring, QA
H	Change	Wie halten wir aktuell?	Updates, Changes